CSAEC510 · PROMPT ENGINEERING · CIE-1

Last-Minute Question Bank

CIE-1 portion only. 75 MCQs (25 per unit) with an answer key and reasons, plus the most probable 5–6 mark questions with model answers, mark splits and keywords.

Unit I · 25 MCQs + 7 long | Unit II · 25 MCQs + 7 long | Unit III · 25 MCQs + 6 long

Scope: Unit I (five principles, text generation models, vectors, Transformer, probabilistic generation) · Unit II (all of Session 1 & 2) · Unit III (intro to LangChain, environment setup, chat models).
Note: questions are predictions based on the syllabus, your slides and the textbook, not a leaked paper.

READ FIRST

How to Use This in the Last Hours

1

Self-test the MCQs

Cover the answer key, do 25 MCQs per unit (~10 min each), then check. Re-read the reason for every wrong one.

2

Read the ★★★ questions

Read every "Very likely" model answer once. Say the definition aloud and look at the keywords.

3

Practise 4 diagrams

Five-principles flow, self-attention, YAML filter chart, LangChain six modules. Draw each once from memory.

MCQ instructions (6 marks)

- Read **all four options** before choosing. Several options often look half-right.
- Watch for **NOT / EXCEPT** in the question. Underline it.
- Eliminate the obviously wrong options first, then compare the remaining two.
- Classic traps: **direction vs format**, **blind prompting vs blind evaluation**, **zero/one/few-shot**, **BERT = encoder / GPT = decoder**, **\s* vs \s+**, **json.loads vs json.dumps**, **Memory vs Callbacks**.
- Never leave an MCQ blank unless your paper has negative marking.

Long-answer instructions (5–6 marks)

- Start with a **1–2 line definition** (examiners look for it first).
- Follow the **mark split** shown under each question. It tells you how much to write for each part.
- Use **tables** for compare/benefits questions and **flow diagrams** for processes. A neat diagram is about 1 mark.
- Include the **keywords** listed under each answer. Underline them in your paper.
- Give **one concrete example**: a prompt, a code line or the shoe/Disney/MongoDB examples.
- Time guide: about **8–10 minutes** for a 5-mark answer and **10–12 minutes** for a 6-mark answer.

PROBABILITY LABELS

★★★ **Very likely** = core syllabus heading with a clear textbook answer. ★★ **Likely** = important but more often asked as a short note or MCQ.

I-A Unit I — 25 MCQs

1. Prompt engineering is best described as:
(a) Training an LLM from scratch
(b) Changing a model's internal weights
(c) Designing and improving instructions to get the desired output from an AI
(d) Writing the AI's source code

2. Which of these is NOT one of the five principles of prompting?
(a) Give Direction
(b) Raise the Temperature
(c) Evaluate Quality
(d) Divide Labor

3. "Generate a shoe name in the style of Steve Jobs" mainly applies:
(a) Give Direction (persona)
(b) Specify Format
(c) Divide Labor
(d) Evaluate Quality

4. "Return only the names, one per line, with no numbers or explanations" applies:
(a) Provide Examples
(b) Divide Labor
(c) Give Direction
(d) Specify Format

5. "A business meeting in Minecraft" (rather than "in the style of Van Gogh") is an example of:
(a) Direction
(b) Format
(c) Evaluation
(d) Task decomposition

6. A prompt that contains no examples at all is called:
(a) One-shot
(b) Few-shot
(c) Zero-shot
(d) Meta prompt

7. Including 2–3 diverse examples of good answers in a prompt is called:
(a) Few-shot prompting
(b) Role drift
(c) Chunking
(d) Blind prompting

8. What is the main risk of adding 20 very similar examples to a prompt?
(a) Higher creativity
(b) The AI copies the pattern, so creativity drops
(c) The output becomes invalid JSON
(d) Responses get shorter

9. Judging a prompt only by looking at one answer and saying "looks good" is called:
(a) Blind evaluation
(b) A/B testing
(c) Blind prompting (trial and error)
(d) Self-evaluation

10. Shuffling responses so the rater does not know which prompt produced each one is:
(a) Chunking
(b) Meta prompting
(c) Progressive summarization
(d) Blind evaluation

11. Prompt A got 3 good responses out of 5; Prompt B got 4 out of 5. Their scores are:
(a) 60% and 80%
(b) 30% and 40%
(c) 3% and 4%
(d) 75% and 80%

12. The evaluation criterion that measures how long the model takes to respond is:
(a) Cost
(b) Latency
(c) Similarity
(d) Refusals

13. BLEU and ROUGE scores are used to evaluate:
(a) Cost
(b) Safety
(c) Similarity to a reference answer
(d) Latency

14. Breaking a complex task into smaller steps (generate → rate → select) is called:
(a) Task decomposition
(b) Tokenization
(c) Quantization
(d) Persona prompting

15. Using one AI to write a prompt for another AI (e.g. an image model) is called:

(a) AI chaining
(b) Meta prompting
(c) Role prompting
(d) ELI5

16. Setting a LOW temperature makes the output:

(a) More random
(b) More creative
(c) More predictable and consistent
(d) Always longer

17. How does an LLM generate its answer?

(a) The whole sentence at once
(b) Paragraph by paragraph
(c) By searching the internet each time
(d) One token at a time, predicting what comes next

18. Which statement about tokens is TRUE?

(a) A token is always one full word
(b) A word like "playing" can be split into tokens such as "play" + "ing"
(c) Punctuation is never a token
(d) All tokenizers split text the same way

19. BPE stands for:

(a) Byte Pair Encoding
(b) Binary Pattern Extraction
(c) Basic Prompt Engineering
(d) Bidirectional Positional Embedding

20. The main benefit of BPE tokenization is that it:

(a) Removes punctuation
(b) Translates text
(c) Handles rare or new words by splitting them into known sub-word pieces
(d) Makes the model larger

21. In a word-embedding space, words with similar meanings:

(a) Are placed far apart
(b) Always have identical vectors
(c) Are not represented
(d) Have vectors that are close together

22. The Transformer architecture was introduced in the paper:

(a) "Deep Residual Learning"
(b) "Attention Is All You Need" (2017)
(c) "ImageNet Classification"
(d) "Generative Adversarial Nets"

23. In "The animal didn't cross the road because it was tired", linking "it" to "animal" is done by:

(a) Self-attention
(b) Quantization
(c) Tokenization
(d) Positional pruning

24. Which pairing is correct?

(a) BERT = decoder; GPT = encoder
(b) Both are decoders
(c) BERT = encoder (understanding); GPT = decoder (generation)
(d) Both are encoders

25. $w_{next} = \arg\max P(w \mid w_1, \ldots, w_m)$ means:

(a) Choose a random word
(b) Choose the longest word
(c) Choose the most recent word
(d) Choose the token with the highest probability given the previous tokens

Answer key with reasons

1 → (c) It is about the **instructions** (prompt), not training or weights.

2 → (b) The five: Direction, Format, Examples, Evaluate, Divide. Temperature is a model setting.

3 → (a) Referencing a persona/style = **direction**.

4 → (d) It defines how the output must **look** = format.

5 → (b) Minecraft = the **type/form** of output (format). Van Gogh = style (direction). Classic trap.

6 → (c) Zero-shot = 0 examples; one-shot = 1; few-shot = several.

7 → (a) Providing examples = few-shot prompting.

8 → (b) More examples → more consistency but **less creativity**.

9 → (c) Trap: **blind prompting** = no system; **blind evaluation** = shuffled, unbiased rating.

10 → (d) Hides the source → removes unconscious bias.

11 → (a) 3/5 = 60%, 4/5 = 80% → Prompt B is better (A/B test).

12 → (b) Latency = response time (1 s vs 10 s).

13 → (c) They compare the AI answer with an expected answer.

14 → (a) Principle 5, Divide Labor = task decomposition.

15 → (b)AI → creates prompt → another AI = meta prompting.

16 → (c) Low temp ≈ almost always the top token; high temp = varied.

17 → (d) Prompt → predict next token → repeat.

18 → (b) Token ≠ word. Exact tokens depend on the tokenizer.

19 → (a)BPE merges frequent character pairs into larger tokens.

20 → (c) Unknown words are broken into pieces the model already knows.

21 → (d)e.g. king–queen close; apple–orange close.

22 → (b) Its key innovation is the attention mechanism.

23 → (a) Each token decides which other tokens matter to it.

24 → (c)BERT understands text; GPT generates one token at a time.

25 → (d) This is the decision mechanism of text generation.

I-B Unit I — Most Probable 5–6 Mark Questions (7)

Q1. Explain the five principles of prompting with suitable examples.

6 MARKS ★★★ VERY LIKELY

½ · Definition | 4 · 5 principles with example | 1 · Diagram | ½ · Conclusion

Prompt engineering is the process of designing and improving prompts to get useful, reliable and consistent output from an AI. The textbook gives five timeless principles (example: naming a shoe that fits any foot size):

Principle	Meaning	Example
1. Give Direction	Describe the desired style in detail or reference a persona	"...in the style of Steve Jobs" → simple, memorable names
2. Specify Format	Define the rules and required structure of the response	"Return the results as a comma-separated list" / "Return only JSON"
3. Provide Examples	Insert diverse examples of correct answers (few-shot)	Refrigerator → iBarFridge; Watch → iNaut → AI outputs iFitFoot
4. Evaluate Quality	Identify errors and rate responses to see what works	A/B test: Prompt A 3/5 = 60%, Prompt B 4/5 = 80% (blind evaluation)
5. Divide Labor	Split complex tasks into multiple steps / chained prompts	Generate 20 names → rate each → select top 5



Conclusion: Good Prompt = Clear Task + Style + Format + Examples + Constraints → consistent, measurable results.

KEYWORDS TO INCLUDE: five principles | persona | few-shot | A/B testing | task decomposition | reliable output

INSTRUCTION

Draw the 5-box flow on top of your answer. If time is short, write the table only. It alone covers 5 of the 6 marks.

Q2. What is a naive prompt? Explain its limitations and how prompt engineering overcomes them.

5 MARKS ★★★ VERY LIKELY

1 · Definition + example | 4 · 5 problems + fixes

A **naive prompt** asks the AI for something without giving many instructions, e.g. “Give me names for shoes that fit any foot size.” It works for casual use but not for a real product used by thousands of people.

Problem	What goes wrong	Fix
1. Vague direction	AI doesn’t know the style (short? modern? funny?)	Give Direction: “10 short, modern, easy-to-pronounce names”
2. Unformatted output	Different format each time; code can’t read it	Specify Format: “one name per line, no numbers”
3. Missing examples	AI can’t see what “good” looks like	Provide Examples: Nike, Adidas, Puma → few-shot
4. Limited evaluation	No way to measure which answer is best	Evaluate Quality: weighted criteria / thumbs up-down
5. No task division	Everything asked in one step	Divide Labor: generate → check → score → improve

KEYWORDS TO INCLUDE **naive prompt** **vague** **unformatted** **few-shot** **evaluation** **task division**

Q3. Explain how the quality of prompts is evaluated. Describe blind evaluation and the criteria used.

6 MARKS ★★★ VERY LIKELY

1 · Why evaluate | 2 · Process / steps | 1 · Blind evaluation | 2 · Criteria

Judging by "looks good" is **blind prompting** (trial and error). An **eval** is a systematic way to measure how well a prompt performs.

Process (A/B test from the textbook)

1. Write two prompts: A = zero-shot, B = few-shot.
2. Run each 5 times (`num_tests = 5`) → 10 responses.
3. Store them in a Pandas DataFrame and save to `responses.csv`.
4. **Shuffle** the responses (blind evaluation).
5. Rate each 👍 = 1 / 👎 = 0.
6. Score: A = 3/5 = 60%, B = 4/5 = 80% → B is better.

Blind evaluation: the rater does not know which prompt produced a response, so there is no unconscious bias. Run several times because LLM output varies between runs.

Human vs automatic: human rating is accurate but slow and costly; automatic checks (expected = "Healthy", AI = "Healthy" ✓) are fast at scale.

Criteria: cost, latency, number of calls, performance (accuracy), classification, reasoning, hallucination, safety, refusals, adversarial testing (prompt injection), similarity (BLEU, ROUGE, vector similarity).

KEYWORDS TO INCLUDE **blind prompting** **eval** **A/B test** **shuffle** **blind evaluation** **latency** **hallucination** **BLEU/ROUGE**

TRAP

Don't mix up **blind prompting** (no system, just "looks good") with **blind evaluation** (shuffled, unbiased rating). Examiners use this as a trap.

Q4. Explain the principle of Divide Labor with task decomposition, meta prompting and AI chaining.

5 MARKS ★★ LIKELY

1½ · Definition + why | 1 · Example | 2 · Meta prompting / chaining | ½ · Diagram

Divide Labor = break a complex task into smaller steps instead of one huge prompt (**task decomposition**). Why: a big prompt is confusing, less predictable, hard to debug and more likely to hallucinate. Small steps show **where** a problem occurred.



- **Self-evaluation:** once the answer is complete, the AI can judge the whole output: Generate → Review → Improve.
- **Meta prompting:** AI writes a prompt for another AI. Describe the shoe → convert to an image prompt → image model → image.
- **AI chaining:** product idea → names → evaluate → select → describe → image prompt → image. LangChain builds such chains.
- **Progressive summarization:** split document → summarise chunks → combine → summarise again.

KEYWORDS TO INCLUDE **task decomposition** **self-evaluation** **meta prompting** **AI chaining** **LangChain** **progressive summarization**

Q5. What are tokens and tokenization? Explain BPE and vector representations (word embeddings).

6 MARKS ★★★ VERY LIKELY

1½ · Token + tokenization | 1½ · BPE / types | 2 · Vectors + embeddings | 1 · Diagram

A **token** is the basic unit of text an LLM processes: a word, part of a word, a character or punctuation. Token ≠ word ("playing" → play + ing). **Tokenization** = breaking text into tokens.



BPE (Byte Pair Encoding) starts with characters and **merges frequently occurring pairs** (a + p → ap → app → apple). It handles rare/new words by splitting them into known pieces. Other tokenizers: **WordPiece** (BERT), **SentencePiece** (no need for word boundaries).

Vector representation: computers can't read meaning, so each token becomes a list of numbers, $v = [v_1, \dots, v_n]$, e.g. "cat" → [0.21, −0.43, 0.75, ...]. A **word embedding** captures meaning and relationships: similar words have nearby vectors (king–queen, walk–walking, apple–orange).

Tokens also define the **context window**, the maximum text a model can process at once (≈ 100 tokens ≈ 75 English words).

KEYWORDS TO INCLUDE **token ≠ word** **tokenizer** **BPE** **rare words** **WordPiece** **vector** **embedding** **semantic similarity** **context window**

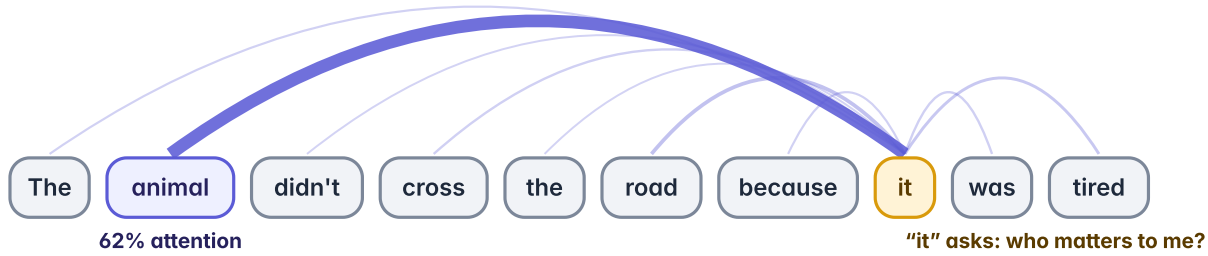
Q6. Explain the Transformer architecture and self-attention. Differentiate between BERT and GPT.

6 MARKS ★★★ VERY LIKELY

1 · Transformer intro | 2 · Self-attention + example | 1 · Components | 2 · BERT vs GPT

The **Transformer** (paper "Attention Is All You Need", 2017) is the architecture behind modern LLMs. Earlier **RNNs** read text one word at a time, which made links between distant words hard to learn. Transformers let every token look at every other token directly (and in parallel).

Self-attention: each token asks "which other tokens are important for understanding me?" In "The animal didn't cross the road because it was tired", attention links **it ↔ animal**. It works like voting: in "deposited money in the bank", bank = financial institution; in "sat near the river bank", bank = river side. This handles **long-range dependencies** ("Ravi … he").



Components: input embedding + positional encoding → multi-head attention → add & norm → feed forward (block repeated × N) → linear → softmax → output probabilities.

	BERT	GPT
Part used	Encoder	Decoder
Purpose	Understanding text	Generating text
Reads	Whole input, both directions	Only previous tokens (masked attention)

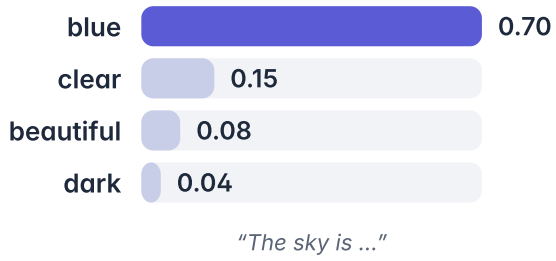
KEYWORDS TO INCLUDE **Attention Is All You Need** **self-attention** **context** **long-range dependencies** **multi-head attention** **positional encoding** **encoder** **decoder**

Q7. What are text generation models? Explain probabilistic text generation (the decision mechanism).

5 MARKS ★★ LIKELY

1 · Definition | 2 · Example + formula | 1 · Loop diagram | 1 · Temperature

A **text generation model** generates new text from its input: Input → understand context → predict the next token → generate text (ChatGPT uses LLMs this way).



$$w_{\text{next}} = \arg\max_w \mathrm{P}(w \mid w_1, \ldots, w_{\text{m}})$$

Choose the token with the **highest probability given the previous tokens**, append it, then repeat until the answer is complete.

Temperature adds controlled randomness: low → predictable (always "blue"); high → creative ("clear", "beautiful"). Because probabilities depend on the context, a small change in the prompt can change the whole output ("AnyFit ___" → Athletic 72.35%).

KEYWORDS TO INCLUDE **next token** **probability distribution** **arg max** **temperature** **context** **repeat loop**

II-A Unit II — 25 MCQs

1. Which did the optimized Disney-list prompt NOT do?

(a) Limit the list to five
(b) Return only male characters
(c) Remove any commentary
(d) Include the film name for each character

2. Letting the model solve and check internally but showing the student only a hint is the:

(a) Inner monologue tactic
(b) ELI5 tactic
(c) Role prompting tactic
(d) Chunking tactic

3. Adding the word "hierarchical" to an outline prompt tells GPT to:

(a) Be more creative
(b) Give main headings and sub-headings, not a flat list
(c) Return JSON
(d) Translate the outline

4. If you ask GPT for exactly 10 headings, it:

(a) Always returns exactly 10
(b) Returns at most 5
(c) May return a different number, so your code should validate
(d) Raises an error

5. In this context, parsing means:

(a) Generating new text
(b) Translating text
(c) Counting tokens
(d) Converting unstructured text into structured data

6. In the regex `r'\* (.+)'`, `\*` matches:

(a) A literal asterisk (*)
(b) Any character
(c) Zero or more of the previous item
(d) A whitespace character

7. In regex, `(.+)` means:

(a) A literal dot followed by a plus
(b) Capture one or more of any character
(c) An optional character
(d) Digits only

8. `\s*` differs from `\s+` because `\s*` matches:

(a) One or more letters
(b) Only tabs
(c) Zero or more whitespace characters
(d) Exactly one space

9. `re.findall()` returns:

(a) Only the first match
(b) The number of matches
(c) True or False
(d) A list of all matches

10. The main limitation of Example 3-1 (two `findall` calls) is that it:

(a) Cannot find headings
(b) Loses which sub-heading belongs to which heading
(c) Only works on JSON
(d) Is too slow

11. Example 3-2 keeps the outline structure by storing results in a:

(a) Dictionary (heading → list of sub-headings)
(b) Single string
(c) CSV file
(d) Set

12. According to the textbook, regular expressions should be avoided when:

(a) The pattern is simple
(b) You only need headings
(c) The data becomes complex or nested
(d) The output is short

13. Which function converts a JSON string returned by GPT into a Python dictionary?

(a) json.dumps()
(b) re.findall()
(c) yaml.dump()
(d) json.loads()

14. Why can `json.loads()` fail on "Sure! Here's the JSON: ```json {...}```"?

(a) The extra text and code fences are not valid JSON
(b) JSON cannot store lists
(c) json.loads only reads files
(d) GPT cannot write JSON

15. An advantage of YAML over JSON is that YAML:

(a) Is always faster
(b) Supports comments using #
(c) Needs more brackets and quotes
(d) Can only store images

16. In YAML payload filtering, if none of the user's items exist in the schema, GPT should return:

(a) An empty JSON object
(b) All the items anyway
(c) "No Items"
(d) An error message

17. Mermaid code generated by GPT is used to:

(a) Style text
(b) Compress data
(c) Translate languages
(d) Create diagrams/flowcharts from text

18. The ELI5 technique asks GPT to:

(a) Explain using very simple words and examples
(b) Answer in five sentences
(c) List five items
(d) Use five examples

19. LLM translation quality is generally weaker for:

(a) English
(b) Low-resource languages with little training data
(c) Spanish
(d) Popular programming languages

20. "Should I use MongoDB or PostgreSQL? First, tell me what information you need" is an example of:

(a) Role prompting
(b) Chunking
(c) Asking for context
(d) Self-evaluation

21. Which is NOT one of the textual features used in text style unbundling?

(a) Tone
(b) Vocabulary
(c) Structure
(d) Temperature

22. Splitting "The local council increased the education budget" into single words is poor chunking because:

(a) It causes loss of context and meaning
(b) It is too fast
(c) It reduces tokens
(d) It creates JSON

23. Which chunking strategy gives accurate token counts (e.g. using tiktoken)?

(a) Splitting by paragraph
(b) Splitting by tokens
(c) Splitting by topic
(d) Splitting by complexity

24. In role prompting, "role drift" means the AI:

(a) Refuses to take any role
(b) Takes several roles at once
(c) Gradually stops following its assigned role in a long conversation
(d) Becomes more accurate

25. "Answer only using the provided article. If the answer isn't there, say 'I could not find an answer'" is used to:

(a) Increase creativity
(b) Chunk text
(c) Translate text
(d) Avoid hallucinations using reference text

Answer key with reasons

1 → (d) It said "Never include the film for each Disney character."

2 → (a) Intermediate steps hidden; only the hint is shown.

3 → (b) Hierarchical = arranged in levels.

4 → (c) "Asking for 10 doesn't guarantee 10."

5 → (d) Text → Python list/dict.

6 → (a) `*` is special in regex, so it is escaped with `\`.

7 → (b) `.` any char, `+` one or more, `( )` capture.

8 → (c) `\s*` = 0+; `\s+` = 1+. Used in Example 3-2 for flexibility.

9 → (d) findall = find ALL.

10 → (b) It produces two separate lists.

11 → (a) Uses `current_section` and `result_dict`.

12 → (c) Then ask GPT for JSON/YAML instead.

13 → (d) loads = load from string.

14 → (a) Tell GPT "Only return valid JSON" and use try/except.

15 → (b) YAML: fewer special characters, readable, comments.

16 → (c) Figure 3-1: No → "No Items"; Partial → filtered; All valid → all.

17 → (d) A renderer turns `graph TD …` into a flowchart.

18 → (a) "Explain it like I'm five."

19 → (b) More training data → better translations.

20 → (c) Let GPT ask for missing details before deciding.

21 → (d) Features: Tone, Length, Vocabulary, Structure, Content.

22 → (a) Also increases processing (many tiny pieces).

23 → (b) Disadvantage: adds computational complexity.

24 → (c) Challenges: bias/stereotypes, role drift, incorrect information.

25 → (d) Restricts the answer to the given source.

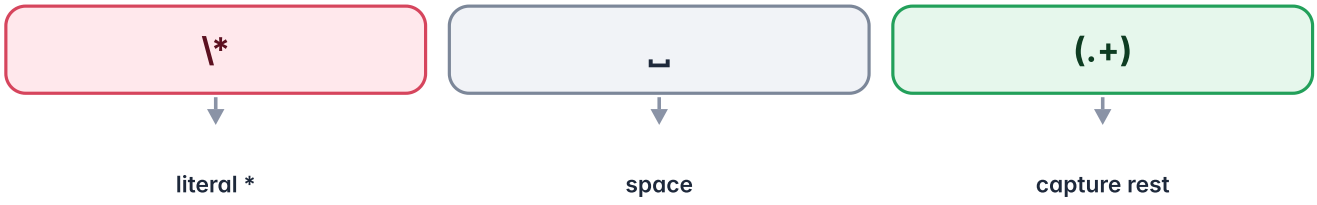
II-B Unit II — Most Probable 5–6 Mark Questions (7)

Q1. Explain hierarchical list generation and how the output is parsed using regular expressions (Examples 3-1 and 3-2).

6 MARKS ★★★ VERY LIKELY

1 · Hierarchical list + prompt | 2 · Parsing + patterns | 1½ · Code / output | 1½ · Limitation → 3-2

A **hierarchical list** is a nested output (headings → sub-headings), e.g. an article outline. The prompt uses "**hierarchical**", "**incredibly detailed**" and an **example structure** (`* Heading` , `a. sub-heading`). **Parsing** = converting this unstructured text into structured data.



Heading pattern r' (.+)' → "Introduction"*



Sub-heading pattern r'\s+[a-z]\. (.+)'

```
headings = re.findall(r'\* (.+)', openai_result)          # Example 3-1
subheadings = re.findall(r'\s+[a-z]\. (.+)', openai_result)
```

Limitation of 3-1: two separate lists, so you can't tell which sub-heading belongs to which heading. **Example 3-2** loops line by line with `current_section` . A heading creates `result_dict[heading] = []` ; each sub-heading is appended to the current heading. It uses `\s*` (0+ spaces) for flexibility and `group(1)` for the captured text. Result: `{"Introduction": ["a. …", "b. …"], …}` .

KEYWORDS TO INCLUDE **hierarchical** **parsing** **regex** *** literal** **(.+) capture** **findall** **dictionary** **current_section** **group(1)**

INSTRUCTION

Write the regex pattern and draw an arrow under each symbol with its meaning. This alone earns 2 marks. Asking for 10 headings doesn't guarantee 10, so mention validating the count.

Q2. When should you avoid regular expressions? Explain how to generate JSON with ChatGPT and handle common problems.

5 MARKS ★★★ VERY LIKELY

1 · When to avoid regex | 1½ · JSON prompt | 1 · json.loads | 1½ · Problems

Regex works for **simple patterns**, but as data becomes complex or nested it becomes hard to write and maintain. Instead ask GPT for a structured format: JSON, YAML, CSV, Mermaid or code.

PROMPT

```
Compose a detailed article outline on "The benefits of learning code".
Only return valid JSON.
Here is an example of the JSON structure:
{"Introduction": ["a. …", "b. …"], "Conclusion": ["a. …", "b. …"]}
```

This is **direction + examples**. Then `json.loads(gpt_output)` turns the JSON string into a Python dictionary, e.g. `data["Introduction"]`.

Problems: (1) extra text or code fences ("Sure! Here's the JSON: ```json ...```") break `json.loads`; (2) **invalid JSON** like `{"Age":}` raises an error. So always say "only return valid JSON" and wrap the parsing in `try/except` (retry on failure).

Regex	JSON
You write custom patterns	GPT returns the structure
Breaks if the format changes	One line: json.loads()
Good for simple patterns	Better for structured/nested data

KEYWORDS TO INCLUDE **complex data** **only return valid JSON** **example structure** **json.loads** **dictionary** **invalid JSON** **try/except**

Q3. What is YAML? Compare it with JSON and explain filtering YAML payloads.

6 MARKS ★★★ VERY LIKELY

1 · Definition + example | 1½ · Advantages vs JSON | 2 · Filtering process | 1½ · Diagram

YAML stores the same structured data as JSON in an indentation-based, human-readable form.

JSON

```
{"item": "Apple", "quantity": 5, "unit":
"pieces"}
```

YAML

```
# shopping item
item: Apple
quantity: 5
unit: pieces
```

Advantages: ① fewer special characters (`{ } [ ] , "`) ② readability ③ comments with `#`.

Filtering: give GPT a **schema** of allowed items (Apple Slices, Milk, Bread, Eggs) and the user query ("5 apple slices, 2 dozen eggs, 3 bananas"). GPT returns YAML with only the allowed items (bananas removed). If nothing matches → **"No Items"**. The LLM acts as a reasoning engine/filter instead of if/else code, but output should still be validated with code.

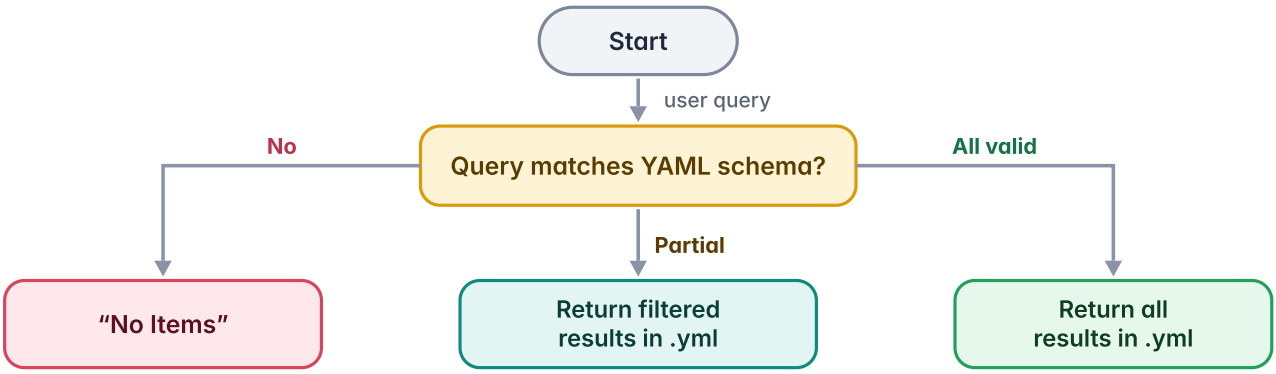


Figure 3-1: using an LLM to decide the control flow of an application instead of code

KEYWORDS TO INCLUDE indentation readability comments schema No Items reasoning engine validate

Q4. Explain text style unbundling: identifying textual features and generating new content.

6 MARKS ★★★ VERY LIKELY

1 · Definition | 1 · Benefits | 2½ · 5 features | 1½ · New content / extraction

Text style unbundling = identifying the writing characteristics of a text so that new content can be written in the same style. Example: "AI is changing our world... we must use it responsibly" → tone positive and informative, short, simple vocabulary, intro → benefit → warning.

Benefits: brand consistency (100 writers, one style), faster content creation, adaptability (blogs, emails, ads).

Feature	Example
Tone	formal / casual / funny / professional
Length	short 100 · medium 500 · long 2000 words
Vocabulary & phrasing	simple "works quickly" vs technical "high computational efficiency"
Structure	title → intro → main points → examples → conclusion
Content	explain → benefits → risks → ethical use

Generating new content: original text → analyse style → extract features → style guide → GPT → "Write an article about **cybersecurity** using the same tone, length, vocabulary and structure." **Extracting specific features:** ask for entities, names, dates, sentiment, keywords ("AI, healthcare, entertainment").

KEYWORDS TO INCLUDE **style guide** **tone** **length** **vocabulary** **structure** **content** **brand consistency** **entity extraction**

Q5. What is chunking? When should it be used? Explain the chunking strategies with advantages and disadvantages.

6 MARKS ★★★ VERY LIKELY

1 · Definition + why | 1 · Use / don't use | 3 · Strategies table | 1 · Poor vs good

Chunking = breaking a large text into smaller meaningful pieces. Needed because LLMs have a limited **context length**: too many tokens → the request is rejected or cut off.

Use for: large documents (500-page book), complex analysis, multi-topic documents. **Avoid for:** short documents, simple analysis, single-topic text. **Poor chunking** splits by word ("The", "local", "council"...), which loses context and increases processing. Split by sentence instead.

Strategy	Advantage	Disadvantage
Sentence	Preserves context	Inefficient for very long content
Paragraph	Keeps related ideas together	Less granular; misses subtle links
Topic	Finds themes; good for classification	Needs topic identification
Complexity	Groups similar difficulty; adaptive	Needs complexity measurement
Length	Handles very long text efficiently	Cuts sentences → loss of context
Tokens (tiktoken)	Accurate token counts within limits	Adds computational complexity

KEYWORDS TO INCLUDE **context length** **meaningful chunks** **loss of context** **sentence** **paragraph** **topic** **complexity** **length** **tiktoken**

INSTRUCTION

Memory hook for the strategies: **S-P-T-C-L-T** (Sentence, Paragraph, Topic, Complexity, Length, Tokens). Draw Figure 3-5 (text → chunks → topics → aggregate) for an extra mark.

Q6. Explain role prompting: its benefits, challenges and when to use it.

6 MARKS ★★★ VERY LIKELY

1 · Definition + example | 2½ · Benefits | 1½ · Challenges | 1 · When to use

Role prompting = telling the AI to "act as" a particular person or expert. "Review MongoDB" → "**Act as a technology reviewer.** Give an in-depth review of MongoDB including features, advantages, disadvantages and comparisons."

Benefits (6): more focused answers · specific expertise (programming instructor, business analyst) · control of response style (formal, friendly…) · creativity (storyteller) · different perspectives (teacher / engineer / analyst) · better engagement (mock interviewer).

Challenges (3): bias & stereotypes from certain roles · **role drift** (AI slowly stops following the role in long chats) · **incorrect information** (an "expert" role changes style, not reliability).

When to use (5): specific expertise, a specific style, creative responses, different perspectives, engaging conversations.

KEYWORDS TO INCLUDE act as | persona | expertise | response style | perspectives | bias | role drift | incorrect information

Q7. How can hallucinations be avoided? Explain the use of reference text with an example.

5 MARKS ★★★ VERY LIKELY

1 · Hallucination definition | 2 · Reference text + example | 1 · Citations / retrieval | 1 · Diagram

A **hallucination** is output that sounds correct but is unsupported or false. Fix: give **reference text** and restrict the answer to it.

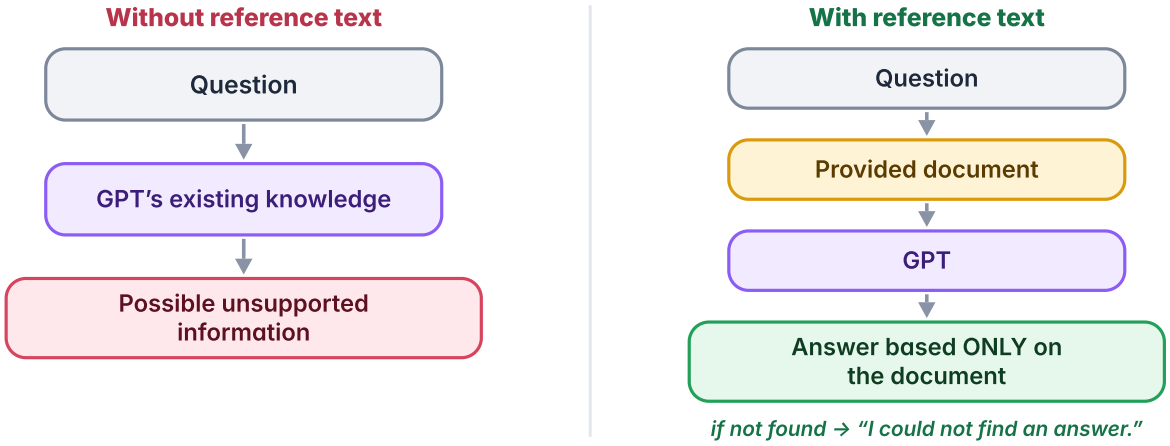
PROMPT

```
Only answer using the provided article. If the answer isn't there, say "I could not
find an answer."
Article: "B2B customers usually have longer sales cycles."
Question: Are B2C sales more cost-effective?
```

OUTPUT

```
I could not find an answer.
```

Citations: ask GPT to include the passage used as evidence (nuclear reactor example), so answers can be verified. **Dynamic retrieval (RAG):** for a 500-page report, search for the relevant sections and send only those to the model. Supporting tactics: thinking time (solve → check → answer) and self-evaluation.



KEYWORDS TO INCLUDE hallucination | reference text | only answer from | I could not find an answer | citations | retrieval / RAG

Also revise for 2–3 mark short notes

Topic	One-line answer
Generating lists	Naive list → wrong format, extra text, uncontrolled count, inconsistent names, no filtering. Fix: count + filter + format + no commentary + example list.
ELI5	"Explain like I'm five": very simple words and examples (cancer example).
Universal translation	LLMs translate languages and complexity levels; meaning can be lost across steps; weaker for low-resource languages.
Ask for context	"What information do you need first?" Loop: context sufficient? → answer : ask → user provides.
Diverse formats	JSON, YAML, CSV, Mermaid (text → diagram), code; mock CSV = fake test data; be clear about the format.
Thinking time / inner monologue	Solve → check → answer; hide intermediate steps and show only a hint (tutoring).
Self-eval	Generate → review → improve (Hello World → type hints, default argument, docstring).

III-A Unit III — 25 MCQs

1. LangChain is:
(a) A large language model
(b) A vector database
(c) A framework for building applications that use LLMs
(d) A tokenizer

2. LangChain is available for:
(a) Python and TypeScript
(b) Java and C++
(c) Python only
(d) R and Julia

3. You cannot summarise a 500-page book in a single prompt mainly because of the:
(a) Temperature
(b) Context length limit
(c) Role drift
(d) Callbacks module

4. Using the output of one LLM call as the input to the next call is a:
(a) Parallel call
(b) Tokenizer
(c) Callback
(d) Sequential workflow (chain)

5. Breaking "plan a 5-day trip with ₹50,000" into budget, transport, hotels, food and activities is:
(a) Quantization
(b) Role prompting
(c) Task decomposition
(d) Retrieval

6. In LangChain, "enhance data awareness" means:
(a) Connecting the LLM to private or external data sources
(b) Increasing model size
(c) Lowering the temperature
(d) Encrypting the API key

7. In LangChain, "enhance agency" means giving the LLM the ability to:
(a) Train itself
(b) Decide what action to take and use tools
(c) Store tokens
(d) Render diagrams

8. Which is NOT one of the six common LangChain modules?
(a) Model I/O
(b) Retrieval
(c) Chains
(d) Tokenizer

9. The module that remembers information between interactions is:
(a) Memory
(b) Callbacks
(c) Model I/O
(d) Retrieval

10. The module that decides which tool or action to use is:
(a) Chains
(b) Memory
(c) Agents
(d) Callbacks

11. The module that runs code when specific events happen is:
(a) Retrieval
(b) Callbacks
(c) Chains
(d) Agents

12. The module used for communicating with the LLM (prompts in, outputs out) is:
(a) Memory
(b) Agents
(c) Chains
(d) Model I/O

13. The module used for finding relevant information (documents, databases) is:
(a) Retrieval
(b) Callbacks
(c) Model I/O
(d) Memory

14. Which command installs LangChain with OpenAI integration using pip?
(a) pip get langchain
(b) pip install langchain langchain-openai
(c) npm install langchain
(d) pip install openai-chain

15. The Conda equivalent is:

(a) conda add langchain
(b) conda get langchain-openai
(c) conda install -c conda-forge langchain langchain-openai
(d) conda run langchain

16. Which command creates a Python virtual environment named venv?

(a) pip install venv
(b) venv create
(c) python venv start
(d) python -m venv venv

17. On Linux/macOS, the virtual environment is activated with:

(a) source venv/bin/activate
(b) activate venv
(c) python venv/activate
(d) pip activate venv

18. To install all the packages listed for the book, you run:

(a) pip install book
(b) pip freeze
(c) pip install -r requirements.txt
(d) pip list requirements

19. The recommended environment variable for the OpenAI key is:

(a) OPENAI_KEY_ID
(b) OPENAI_API_KEY
(c) LANGCHAIN_KEY
(d) GPT_SECRET

20. Which library loads the API key from a `.env` file?

(a) pandas
(b) tiktoken
(c) requests
(d) python-dotenv

21. Passing `api_key="…"` directly into `ChatOpenAI()` is suitable for:

(a) Quick prototyping only
(b) Production apps
(c) Sharing on GitHub
(d) Every project

22. The import used in the book for the OpenAI chat model is:

(a) import openai.Chat
(b) from langchain import GPT
(c) from langchain_openai.chat_models import ChatOpenAI
(d) from openai import LangChain

23. A chat model in LangChain takes as input:

(a) A single integer
(b) A list of messages
(c) A CSV file
(d) A tokenizer

24. The message type that sets the AI's role or persona ("Act as a senior software engineer…") is:

(a) HumanMessage
(b) AIMessage
(c) ToolMessage
(d) SystemMessage

25. In the book's example, `chat.invoke(messages)` returns:

(a) An AIMessage whose text is in `.content`
(b) A Python list of tokens
(c) A JSON file
(d) Nothing

Answer key with reasons

1 → (c) It connects LLMs with data, tools, memory and multiple steps.

2 → (a) Stated directly in your slides.

3 → (b) Book larger than the context window.

4 → (d) e.g. characters → plot → chapters.

5 → (c) Complex reasoning → smaller tasks.

6 → (a) e.g. employee data in a company database.

7 → (b) e.g. travel assistant uses flight/hotel tools.

8 → (d) Six: Model I/O, Retrieval, Chains, Agents, Memory, Callbacks.

9 → (a) Memory = remembering previous conversation.

10 → (c) Agents choose actions/tools.

11 → (b) e.g. logging, streaming, monitoring.

12 → (d) Model input/output.

13 → (a) Retrieval = find relevant information.

14 → (b) langchain = framework; langchain-openai = OpenAI connector.

15 → (c) `-c conda-forge` selects the channel.

16 → (d) Recommended before installing packages.

17 → (a) Windows: venv\Scripts\activate.

18 → (c) `-r` reads a requirements file.

19 → (b) Set in the terminal or a .env file.

20 → (d) `load_dotenv()` reads .env into environment variables.

21 → (a) Hard-coding keys is a security risk.

22 → (c) Then `chat = ChatOpenAI()`.

23 → (b) Returns a message (AIMessage) as output.

24 → (d) Human = user; AI = model reply.

25 → (a) `print(response.content)` prints the reply.

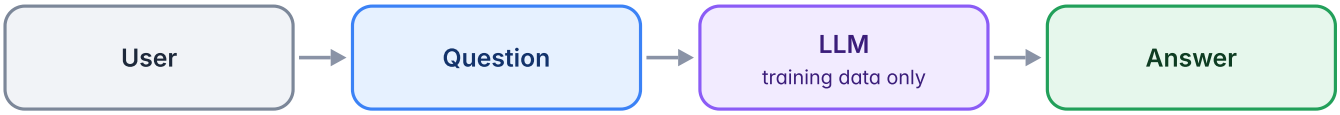
III-B Unit III — Most Probable 5–6 Mark Questions (6)

Q1. What is LangChain? Why is it needed? Explain its two main ideas.

6 MARKS ★★★ VERY LIKELY

1 · Definition | 2 · Need | 1½ · Data awareness | 1½ · Agency

LangChain is a framework for building applications that use LLMs, available in **Python and TypeScript**.



An LLM alone: only its training data, one question → one answer

Need: real applications must read databases, search documents, remember conversations, use tools, perform multiple steps, make decisions and take actions. LangChain **connects the LLM with these components**.

1. Enhance data awareness: connect the LLM to private/external data (e.g. employee database). User question → LangChain → database → relevant info → LLM → answer.

2. Enhance agency: let the LLM decide which action to take and use tools. Travel assistant: search flights → check hotels → calculate cost → create itinerary.

KEYWORDS TO INCLUDE: **framework** | **Python & TypeScript** | **external data** | **data awareness** | **agency** | **tools** | **actions**

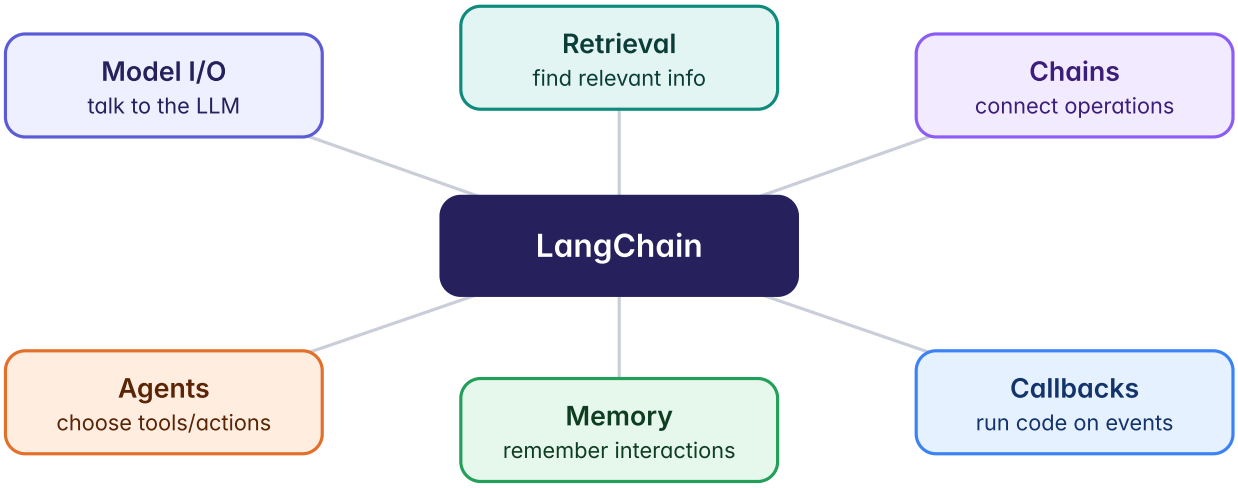
Q2. Explain the six common modules of LangChain.

5 MARKS ★★★ VERY LIKELY

½ · Intro | 3½ · 6 modules | 1 · Diagram

LangChain provides reusable **building blocks**:

Module	Purpose
Model I/O	Communicating with the LLM (prompts in, outputs out)
Retrieval	Finding relevant information (documents, databases)
Chains	Connecting multiple operations in sequence
Agents	Deciding which tool/action to use
Memory	Remembering information between interactions
Callbacks	Running code when specific events happen (logging, streaming)



KEYWORDS TO INCLUDE **Model I/O** **Retrieval** **Chains** **Agents** **Memory** **Callbacks** **building blocks**

INSTRUCTION

Memory hook: **"My Rabbit Can Always Munch Carrots"** (Model I/O, Retrieval, Chains, Agents, Memory, Callbacks).

Q3. Explain the environment setup required for LangChain.

5 MARKS ★★★ VERY LIKELY

1½ · Install | 1½ · Virtual env | 2 · API key

COMMANDS

```
pip install langchain langchain-openai          # or: conda install -c conda-forge langchain
langchain-openai
python -m venv venv                             # create virtual environment
source venv/bin/activate                        # activate it
pip install -r requirements.txt                 # all packages for the book
pip install openai                              # model provider package
```

`langchain` = the framework; `langchain-openai` = lets LangChain talk to OpenAI models. LangChain needs at least one **model provider**.

API key: best practice is to set the OPENAI_API_KEY environment variable, or load it from a `.env` file with **python-dotenv** (`load_dotenv()`). For quick **prototyping only**: `ChatOpenAI(api_key="api_key")`. Hard-coding keys is a security risk.

KEYWORDS TO INCLUDE **pip / conda** **langchain-openai** **virtual environment** **requirements.txt** **OPENAI_API_KEY** **.env** **python-dotenv**

Q4. What are chat models in LangChain? Explain the message types with a code example.

6 MARKS ★★★ VERY LIKELY

1 · Definition | 2 · Message types | 2 · Code | 1 · Explanation

A **chat model** takes a list of messages as input and returns a message (AIMessage). LangChain gives all providers the same interface.

Message	Represents	Purpose
SystemMessage	developer/system	sets role/persona/rules
HumanMessage	user	the question/instruction
AIMessage	model	the reply; kept for history

```
from langchain_openai.chat_models import ChatOpenAI
from langchain.schema import AIMessage, HumanMessage, SystemMessage

chat = ChatOpenAI(temperature=0.5)
messages = [SystemMessage(content="Act as a senior software engineer at a startup company."),
            HumanMessage(content="Please can you provide a funny joke about software
engineers?")]
response = chat.invoke(input=messages)
print(response.content)
```

`temperature` controls randomness; `invoke()` sends the messages; the result is an **AIMessage** whose text is in `.content`. Append the AIMessage and the next HumanMessage to continue the conversation.

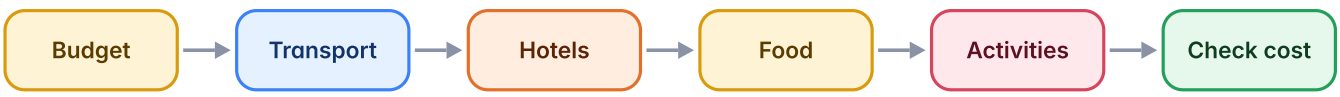
KEYWORDS TO INCLUDE list of messages | SystemMessage | HumanMessage | AIMessage | ChatOpenAI | invoke() | .content | temperature

Q5. Why is basic prompting sometimes not enough? Explain with examples.

5 MARKS ★★ LIKELY

1½ · Context length | 1½ · Sequential calls | 2 · Complex reasoning

- **Context length:** a 500-page book can't fit in one prompt if it exceeds the context window.
- **Combining sequential LLM inputs/outputs:** one call's output feeds the next (create characters Aria, Leo, Mira → plot → chapters) = a chain.
- **Complex reasoning:** "Plan a 5-day trip with ₹50,000" needs budget, transport, accommodation, food, activities, itinerary and a cost check → **task decomposition** into sub-tasks.



Frameworks like LangChain handle these with chains, retrieval and agents.

KEYWORDS TO INCLUDE context length | sequential workflow | chain | complex reasoning | task decomposition

Q6. Differentiate between using an LLM directly and building an application with LangChain.

5 MARKS ★★ LIKELY

4 · Table (5 points) 1 · Conclusion

Aspect	LLM directly	With LangChain
Knowledge	Only its training data	Connects to external/private data (Retrieval)
Steps	One prompt → one answer	Multiple linked calls (Chains)
Memory	Forgets between calls	Remembers conversation (Memory)
Actions	Cannot use tools	Decides and uses tools (Agents)
Monitoring	Manual	Event hooks (Callbacks)
Providers	Code tied to one API	Same interface for many chat models

Conclusion: LangChain adds data awareness and agency on top of the LLM, so it can power real applications.

KEYWORDS TO INCLUDE **training data only** **retrieval** **chains** **memory** **agents** **data awareness** **agency**